

Implementación de un servidor web seguro con certificados SSL autofirmados

Autor: Daniel Monge de Pedro

Fecha de creación: 14/04/2025



Índice

Índice	2
1. Introducción	4
2. Descripción del proyecto	5
3. Tipos de empresas/sectores beneficiados	5
4. Planificación del proyecto	6
4.1 Objetivos y requisitos del proyecto	6
4.2 Viabilidad del proyecto.....	7
4.3 Fases a realizar en el proyecto	7
4.4 Tiempo de realización de las fases del proyecto.....	9
4.5 Prevención de riesgos	10
5. Desarrollo práctico del proyecto	10
5.1 Configuración de adaptadores de red.....	10
5.2 Configuración inicial de máquinas virtuales	13
5.2.1 Servidor	13
5.2.3 Cliente	14
5.3 Instalación y configuración inicial de Ubuntu Server 24.04	15
5.3.1 Actualización del sistema.....	17
5.3.2 Instalación de interfaz gráfica en el servidor	18
5.4 Configuración de los adaptadores de red	19
5.4.1 Servidor web	20

5.4.3 Verificación de conectividad entre máquinas	21
5.5 Instalación y configuración del servidor DNS y DHCP	22
5.5.1 Instalación de dnsmasq.....	22
5.5.2 Configuración del servidor DNS.....	22
5.5.3 Verificación de resolución del dominio y logs.....	24
5.5.4 Creación de servidor DHCP con dnsmasq	26
5.6 Instalación y configuración del servidor web	28
5.6.1 Instalación de Apache2	28
5.6.2 Habilitar módulo SSL y generación de certificado	29
5.6.3 Configuración del Virtual Host	32
5.7 Creación básica de página web para pruebas de acceso	38
5.8 Configuración de firewall con iptables	42
6. Conclusión	46
7. Webgrafía.....	47

1. Introducción

Hoy, la seguridad en los servidores web es fundamental para cualquier empresa que ofrezca servicios en la web, ya sean pequeñas, medianas o grandes. Muchos sitios web no usan HTTPS de forma correcta, ya que, en muchos casos, las configuraciones no son las adecuadas por falta de conocimientos.

Este proyecto me permitirá poner en práctica muchos de los conocimientos adquiridos durante el grado: administración de servidores Linux, configuración de servicios de red, seguridad y configuración de firewalls.

Además, este proyecto resulta útil para entornos de desarrollo o pequeñas empresas que necesiten una solución segura sin depender de servicios de terceros ni realizar inversiones adicionales.

El objetivo principal del proyecto es concienciar de la importancia de la seguridad en la web y demostrar que se pueden implantaciones seguras gratis sin depender de servicios de terceros evitando inversiones que no sean necesarias.

2. Descripción del proyecto

Este proyecto consiste en la implementación de un servidor web seguro y funcional en **Ubuntu Server 24.04**. El servidor contará con certificados **SSL/TLS** autofirmados para tener cifrado **HTTPS**, aunque estos certificados no están emitidos por una entidad certificadora permiten asegurar una conexión cifrada, ideal para entornos de pruebas, o servidores internos.

Además, el servidor estará protegido mediante la configuración de iptables, para asegurar que solo estén abiertos los puertos necesarios asegurando el servidor y dándole una capa extra de seguridad.

Estas configuraciones permitirán entender cómo funciona el cifrado HTTPS, cómo generar y gestionar certificados y como proteger dicho servidor en la red.

3. Tipos de empresas/sectores beneficiados

Este proyecto está orientado a:

- **Departamentos de IT:** los cuales prueban software en entornos locales o de desarrollo.
- **Pequeñas empresas:** que no necesitan una CA oficial por cuestiones de presupuesto o bien porque operan en un entorno cerrado.
- **Centros educativos:** que necesitan entornos de pruebas para realizar prácticas, pero sin tener costes adicionales.

4. Planificación del proyecto

4.1 Objetivos y requisitos del proyecto

Objetivo

El objetivo principal es la instalación y configuración de un servidor web seguro con certificados SSL/TLS autofirmados, con el servidor Apache en Ubuntu server 24.04 y protegerlo con reglas de iptables.

Todo el proyecto se desarrollará con herramientas y software libre.

Requisitos del proyecto

El proyecto tiene varios tipos de requisitos, que son:

Requisitos técnicos:

- Máquina virtual con Ubuntu Server 24.04 instalado.
- Conexión de red (se utilizará una red interna y una conexión puente para el servidor).
- Instalación y configuración de Apache para el servidor web.
- Generación e instalación de certificados SSL autofirmados.
- Configuración de HTTPS en Apache.
- Configuración de iptables.

Requisitos de funcionamiento:

- Asegurar que el servidor responde por HTTPS.
- Asegurar que el certificado funciona de forma correcta.

- Asegurar que el firewall solo permita los puertos configurados, 22(SSH), 443(HTTPS), 53(dnsmasq).
- Asegurar que el servidor DNS configurado con dnsmasq responde correctamente.
- Asegurar que el servidor DHCP con dnsmasq funciona bien.

4.2 Viabilidad del proyecto

Los costes el proyecto no requiere de una inversión económica, ya que en el proyecto se utilizan:

- Entornos virtuales.
- Software libre y totalmente gratuito.
- No depende de servicios de terceros, como certificados de pago.

En cuanto al tiempo estimado de desarrollo de este proyecto podría ser de 3 semanas.

4.3 Fases a realizar en el proyecto

El proyecto se dividirá en varias fases, que son las siguientes:

1. Configuración de máquinas virtuales con VMware:

En esta fase crearé dos máquinas virtuales, una con Ubuntu Server 24.04, que actuará como servidor y otra con Ubuntu 24.04, que actuará como cliente, para posteriormente poder hacer pruebas de funcionamiento.

2. Instalación y configuración de Ubuntu Server 24.04 y configuración básica:

En esta fase del se instala el sistema operativo y se hacen configuraciones básicas, como configuración de red y actualización del sistema.

3. Instalación y configuración del servidor DNS y DHCP:

En esta fase utilizaremos la herramienta dnsmasq , ya que, para este proyecto necesitamos muy pocos dominios, por lo tanto, dnsmasq es una opción excelente, ya que es ligero y flexible.

También utilizaremos dnsmasq , para la configuración de un servidor DHCP.

4. Instalación de Apache y configuración del servidor web:

En esta fase se instalará el servidor web Apache y se configurarán los Virtual Hosts, los cuales sirven para mostrar contenido desde diferentes directorios dependiendo del dominio.

5. Generación e instalación de certificados SSL/TLS autofirmado para el servidor web:

En esta fase se crearán e instalarán los certificados SSL/TLS autofirmados para habilitar HTTPS. Se configurará Apache para que escuche por el puerto 443 y utilice los certificados autofirmados de forma correcta.

6. Configuración de iptables:

En esta fase se configurarán reglas de iptables para permitir únicamente los puertos que sean necesarios, necesarios, que son los puertos 22, 443, 80 y 53.

7. Creación de página web básica con HTML y PHP:

En esta fase se creará una página web básica para cargarla con Apache.

4.4 Tiempo de realización de las fases del proyecto

Tabla 1 – Actividades y tiempo de realización

Fases	Actividad	Duración estimada
1	Configuración de máquinas virtuales	1 día
2	Instalación y configuración básica de Ubuntu Server	1 día
3	Instalación del servidor DNS y DHCP con dnsmasq	1 día
4	Instalación y configuración de Apache	1 día
5	Generación e instalación de certificados SSL/TLS	1 día
6	Configuración de iptables	1 día
7	Creación de página web básica	1 día

4.5 Prevención de riesgos

Para mantener seguro el servidor web en caso de cualquier fallo, se tomarán las siguientes acciones:

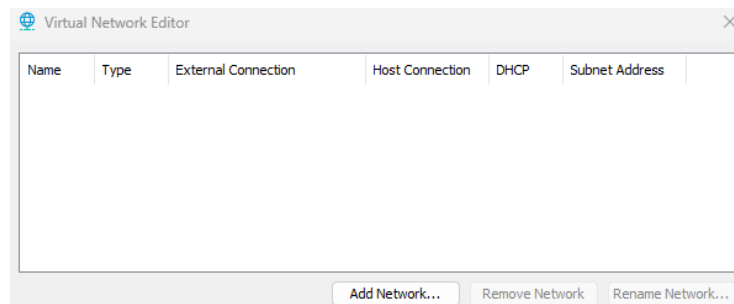
1. Realizar copias de seguridad después de cada cambio importante en la configuración del servidor.
2. Documentar cada paso y cambios realizados, para que, en caso de error, se vea el problema.
3. Ya que estamos trabajando en VMware, se deberían de realizar snapshots, después de hacer cada una de las fases previstas en el proyecto.
4. Monitorizar logs, después de cada configuración importante, para asegurar que todo ha salido de forma correcta.

5. Desarrollo práctico del proyecto

5.1 Configuración de adaptadores de red

Lo primero es abrir el Virtual Network editor:

Imagen 1: Lista de adaptadores de red



En esta captura aparecen los adaptadores de red que hay configurados, vamos a proceder con la configuración de los diferentes adaptadores que necesitaremos.

Vamos a necesitar dos tipos de adaptadores:

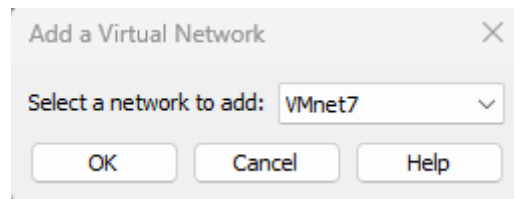
- Un **adaptador puente**, el cual permitirá que el servidor se conecte a la red y así poder instalar y configurar el servidor correctamente.
- Un adaptador que funcionará como **red interna**, ya que el servidor web estará disponible solo en la red que crearemos.

Ahora vamos a proceder con la creación de cada uno de los adaptadores de red:

Adaptador puente

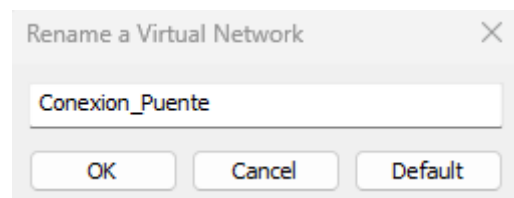
Lo primero le damos a crear un nuevo adaptador de red y elegimos en número de adaptador:

Imagen 2: Elección de número adaptador de red



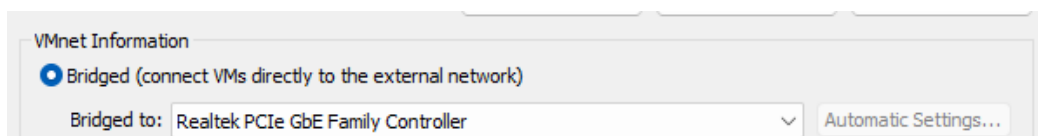
Una vez elegido el número de adaptador, ya saldrá en la lista de adaptadores, ahora procedemos con su configuración, lo primero le vamos a cambiar el nombre:

Imagen 3: Nombre del adaptador



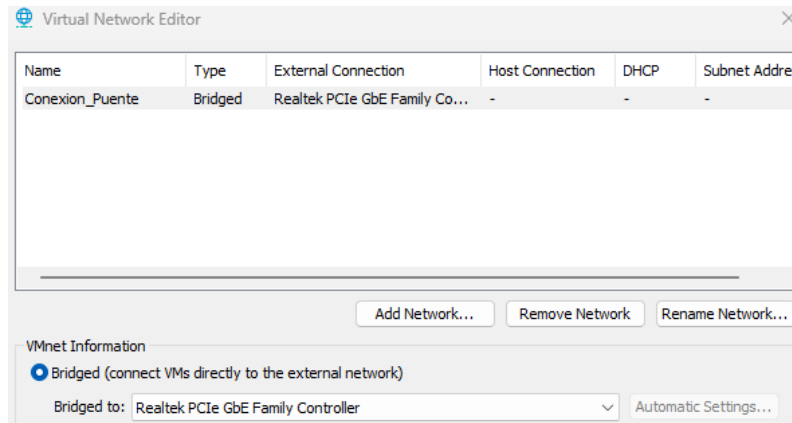
Una vez cambiado el nombre procedemos con la configuración, ahora tengo que elegir el adaptador de red físico que utilizaremos para conectarnos a internet desde la máquina virtual, en este caso estoy por cable, por lo tanto, elegimos el siguiente:

Imagen 4: Nombre del adaptador



Y con esto ya tendríamos el adaptador puente creado:

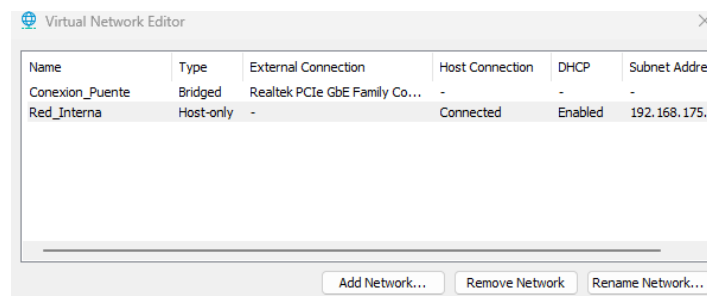
Imagen 5: Muestra de adaptador puente creado



Red Interna

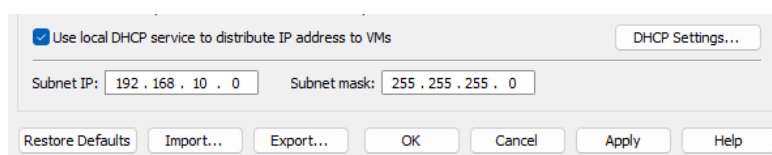
En este adaptador tendríamos que seguir los mismos pasos que el anterior a excepción de la configuración, poniendo el número de adaptador y el nombre, una vez creado el adaptador quedaría así:

Imagen 6: Muestra de adaptador para la red interna creado



Ahora procedemos con su configuración, lo primero es poner la red que vamos a utilizar en esta red interna, que es la siguiente:

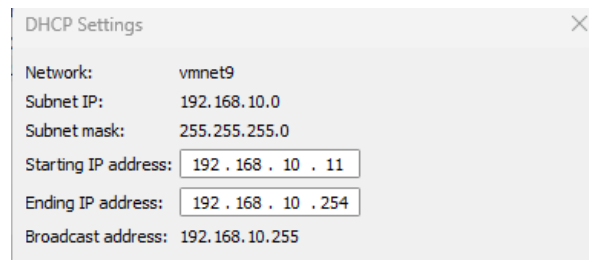
Imagen 7: Dirección IP y máscara de red de la red interna



Ahora dado que el servidor va a tener una IP fija, pero el cliente no, configuraremos un DHCP para este adaptador de red, con el objetivo que a la máquina cliente se le asigne una dirección IP de forma automática al cliente en esta red.

La configuración del DHCP quedaría así:

Imagen 8: Configuración de DHCP para la red interna



En la captura se muestra la configuración del **DHCP**, lo que se ha hecho es dejar libres las primeras 9 IPs para asignar a equipos de forma fija en caso necesario, a partir de esa dirección todas las demás IPs se asignarán de forma dinámica.

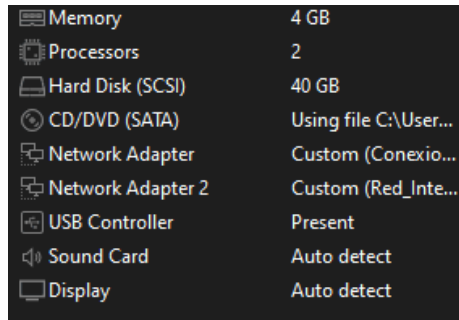
Nota: este DHCP lo he configurado para asignarle una IP al cliente, pero después cuando se configure el DHCP con dnsmasq se deshabilitará para evitar conflictos, ya que con el DHCP de dnsmasq podremos resolver el nombre de dominio que crearemos.

5.2 Configuración inicial de máquinas virtuales

5.2.1 Servidor

La configuración de la máquina que actuará como servidor es la siguiente:

Imagen 9: Configuración inicial de máquina servidor web



Memory	4 GB
Processors	2
Hard Disk (SCSI)	40 GB
CD/DVD (SATA)	Using file C:\User...
Network Adapter	Custom (Conexio...
Network Adapter 2	Custom (Red_Inte...
USB Controller	Present
Sound Card	Auto detect
Display	Auto detect

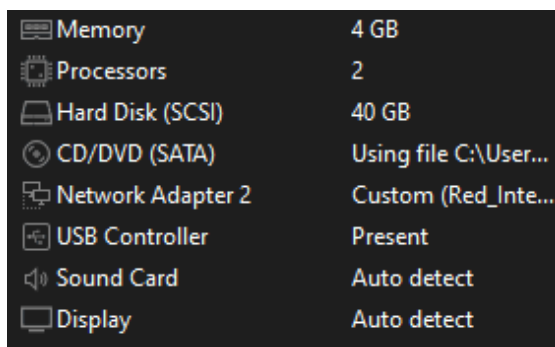
La máquina presenta las siguientes configuraciones iniciales:

1. 4GB de RAM.
2. 2 núcleos del procesador.
3. Disco de 40GB
4. La imagen ISO de Ubuntu Server 24.04.
5. Adaptador puente.
6. Adaptador de red interna.
7. Controlador USB.
8. Auto detecta la tarjeta de sonido de forma automática.
9. Auto detecta la pantalla de forma automática.

5.2.3 Cliente

La configuración de la máquina cliente es la siguiente:

Imagen 11: Configuración inicial de máquina cliente



Memory	4 GB
Processors	2
Hard Disk (SCSI)	40 GB
CD/DVD (SATA)	Using file C:\User...
Network Adapter 2	Custom (Red_Inte...
USB Controller	Present
Sound Card	Auto detect
Display	Auto detect

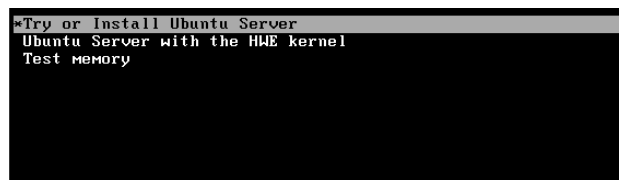
La máquina presenta las siguientes configuraciones iniciales:

1. 2GB RAM.
2. 2 núcleos del procesador.
3. Disco de 20GB.
4. La imagen ISO de Ubuntu 24.04 Desktop.
5. Adaptador de red interna.
6. Controlador USB.
7. Auto detecta la tarjeta de sonido de forma automática.
8. Auto detecta la pantalla de forma automática.

5.3 Instalación y configuración inicial de Ubuntu Server 24.04

Procedemos con la instalación de Ubuntu Server, para ello arrancamos la máquina y la iniciamos con la imagen ISO para empezar con la instalación, una vez iniciada la máquina aparecerá la siguiente pantalla:

Imagen 12: Instalación inicial de Ubuntu Server 24.04



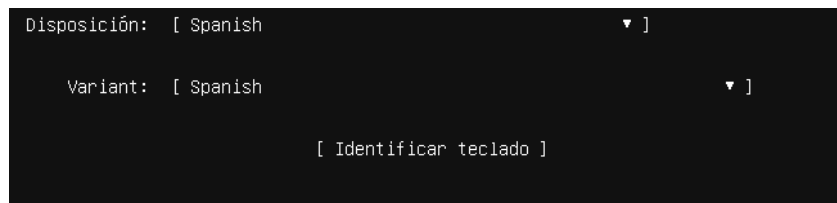
En la captura anterior procedemos con la elección de la primera opción y posteriormente aparecerá en pantalla, la configuración de idioma, en este caso elegiremos el español:

Imagen 13: Elección del idioma



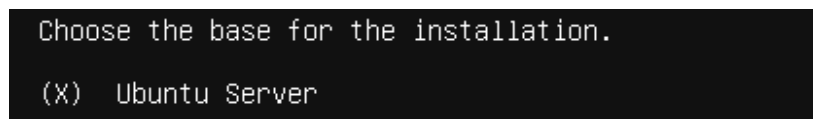
Ahora procedemos con la configuración del teclado, el cual será en español:

Imagen 14: Configuración del teclado



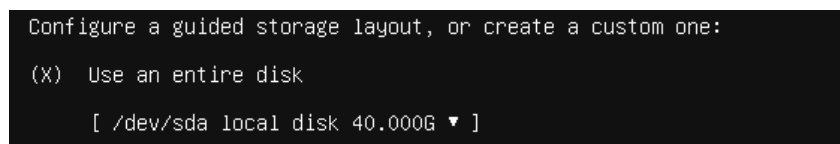
En la siguiente pantalla elegimos, que instale el Ubuntu Server completo:

Imagen 15: Instalación Base



Ahora le indicamos el disco que debe de utilizar para instalar Ubuntu Server:

Imagen 16: Disco para la instalación



Después de elegir el disco en el que se instalará el sistema, debemos poner el nombre del servidor, usuario y contraseña:

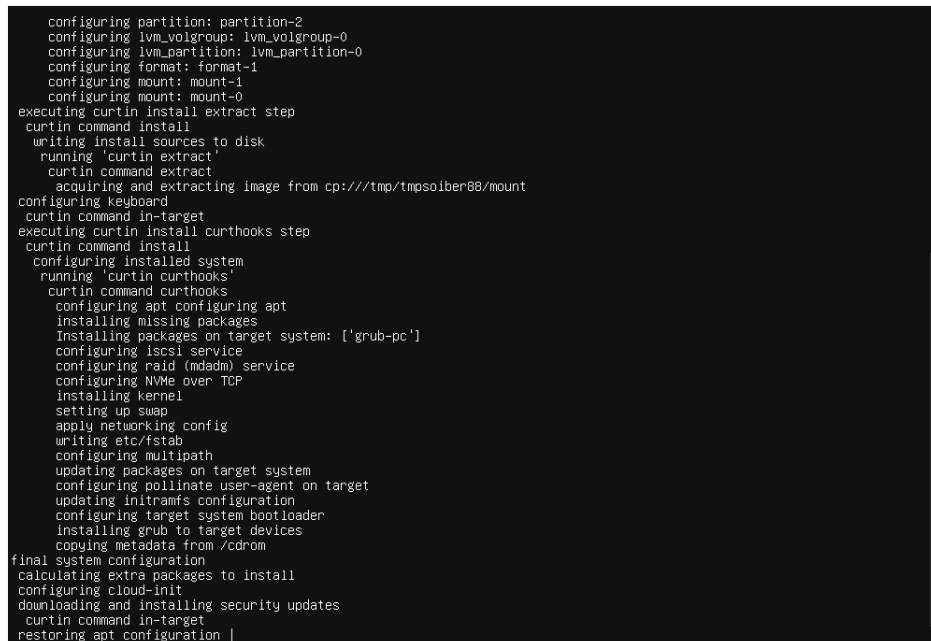
Imagen 17: Nombre del servidor, usuario y contraseña



```
Su nombre: Daniel
Your servers name: daniserver
The name it uses when it talks to other computers.
Elija un nombre de usuario: dani
Elija una contraseña: *****
Confirme la contraseña: *****
```

Al reiniciar empezará la instalación del servidor y esperamos a que se complete:

Imagen 18: Proceso de instalación



```
configuring partition: partition-2
configuring lvm_voigroup: lvm_voigroup-0
configuring lvm_partition: lvm_partition-0
configuring format: format-1
configuring mount: mount-1
configuring mount: mount-0
executing curtin install extract step
curtin command install
writing install sources to disk
running 'curtin extract'
curtin command extract
acquiring and extracting image from cp:///tmp/tmpsoiber88/mount
configuring keyboard
curtin command in-target
executing curtin install curthooks step
curtin command install
configuring installed system
running 'curtin curthooks'
curtin command curthooks
configuring apt
configuring apt
installing missing packages
Installing packages on target system: ['grub-pc']
configuring iscsi service
configuring raid (mdadm) service
configuring NVMe over TCP
installing kernel
setting up swap
apply networking config
writing etc/fstab
configuring multipath
updating packages on target system
configuring pollinate user-agent on target
updating initramfs configuration
configuring target system bootloader
installing grub to target devices
copying metadata from /cdrom
final system configuration
calculating extra packages to install
configuring cloud-init
downloading and installing security updates
curtin command in-target
restoring apt configuration |
```

5.3.1 Actualización del sistema

Ahora entramos dentro del servidor y actualizamos el sistema, para ello primero debemos actualizar los repositorios de paquetes para después poder actualizarlos, utilizaremos el siguiente comando:

Imagen 19: Actualización de repositorios de paquetes

```
dani@daniserver:~$ sudo apt update
[sudo] password for dani:
Obj:1 http://es.archive.ubuntu.com/ubuntu noble InRelease
Obj:2 http://es.archive.ubuntu.com/ubuntu noble-updates InRelease
Obj:3 http://es.archive.ubuntu.com/ubuntu noble-backports InRelease
Des:4 http://es.archive.ubuntu.com/ubuntu noble/main Translation-es [325 kB]
Des:5 http://es.archive.ubuntu.com/ubuntu noble/restricted Translation-es [816 B]
Des:6 http://es.archive.ubuntu.com/ubuntu noble/universe Translation-es [1.371 kB]
Des:7 http://es.archive.ubuntu.com/ubuntu noble/multiverse Translation-es [63,1 kB]
Obj:8 http://security.ubuntu.com/ubuntu noble-security InRelease
Descargados 1.759 kB en 3s (538 kB/s)
Leyendo lista de paquetes... Hecho
Creando árbol de dependencias... Hecho
Leyendo la información de estado... Hecho
Se pueden actualizar 59 paquetes. Ejecute «apt list --upgradable» para verlos.
```

Una vez hecho esto, actualizamos los paquetes disponibles, se utilizará el comando **sudo apt dist-upgrade**, con el objetivo de resolver automáticamente problemas con dependencias.

Ahora escribimos el comando y empezará la instalación, una vez completada reiniciamos el sistema para aplicar los cambios:

Imagen 20: Instalación de actualizaciones

```
Desempaquetando snapd (2.67.1+24.04) sobre (2.66.1+24.04) ...
Preparando para desempaquetar .../67-sosreport_4.8.2-0ubuntu0~24.04.1_amd64.deb ...
Desempaquetando sosreport (4.8.2-0ubuntu0~24.04.1) sobre (4.7.2-0ubuntu1~24.04.2) ...
Preparando para desempaquetar .../68-cloud-init_24.4.1-0ubuntu0~24.04.3_all.deb ...
Desempaquetando cloud-init (24.4.1-0ubuntu0~24.04.3) sobre (24.4-0ubuntu1~24.04.2) ...
Configurando libapparmor1:amd64 (4.0.1really4.0.1-0ubuntu0.24.04.4) ...
Configurando pci.ids (0.0~2024.03.31-1ubuntu0.1) ...
Configurando linux-base (4.5ubuntu9+24.04.1) ...
Configurando python3-problem-report (2.28.1-0ubuntu3.5) ...
Configurando distro-info-data (0.60ubuntu0.3) ...
Configurando update-notifier-common (3.192.68.1) ...
Progreso: [ 54%] [#####]
```

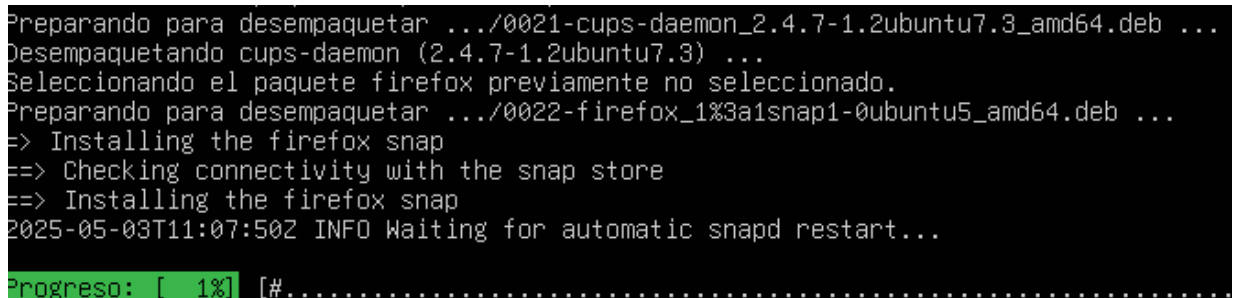
5.3.2 Instalación de interfaz gráfica en el servidor

Una vez ya tenemos actualizado el sistema, procederemos con la instalación de la interfaz gráfica.

En este servidor instalaremos la interfaz gráfica predeterminada de Ubuntu, el cual es **GNOME**, para su instalación, para instalarlo utilizamos el siguiente comando:

sudo apt install ubuntu-desktop -y

Imagen 21: Instalación del entorno de escritorio GNOME



```
Preparando para desempaquetar .../0021-cups-daemon_2.4.7-1.2ubuntu7.3_amd64.deb ...
Desempaquetando cups-daemon (2.4.7-1.2ubuntu7.3) ...
Seleccionando el paquete firefox previamente no seleccionado.
Preparando para desempaquetar .../0022-firefox_1%3a1snap1-0ubuntu5_amd64.deb ...
=> Installing the firefox snap
=> Checking connectivity with the snap store
=> Installing the firefox snap
2025-05-03T11:07:50Z INFO Waiting for automatic snapd restart...
Progreso: [ 1%] [#.....]
```

Una vez introducido el comando empezará la instalación, y al final de esta tendremos que reiniciar el sistema para aplicar cambios y así poder ver el entorno de escritorio de GNOME.

Es importante mencionar que al instalar **ubuntu-desktop**, ya se instala automáticamente el gestor de pantalla **gdm3**.

5.4 Configuración de los adaptadores de red

Antes de comenzar con la configuración de cada servidor, haremos la configuración de sus adaptadores de red, ya que cada servidor debe tener una IP fija en la red.

Para configurar de forma manual las IPs fijas accederemos al archivo de configuración, el cual se encuentra en la ruta:

/etc/netplan/50-cloud-init.yaml**5.4.1 Servidor web**

Lo primero que haremos será identificar el nombre de la interfaz de red, que queremos configurar, para ello haremos uso de **ifconfig**:

Imagen 23: Uso de ifconfig para identificar adaptador

```
ens34: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>  mtu 1500
        inet 192.168.10.12  netmask 255.255.255.0  broadcast 192.168.10.255
        inet6 fe80::20c:29ff:fe1f:a7fd  prefixlen 64  scopeid 0x20<link>
        ether 00:0c:29:1f:a7:fd  txqueuelen 1000  (Ethernet)
        RX packets 1  bytes 342 (342.0 B)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 87  bytes 10362 (10.3 KB)
        TX errors 0  dropped 0  overruns 0  carrier 0  collisions 0
```

Una vez identificado el adaptador de red, realizamos la siguiente configuración en el fichero:

Imagen 24: Fichero de configuración del servidor web

```
GNU nano 7.2 /etc/netplan/50-cloud-init.yaml *
network:
  version: 2
  renderer: networkd
  ethernets:
    ens33:
      dhcp4: true
    ens34:
      dhcp4: no
      addresses:
        - 192.168.10.2/24
      nameservers:
        addresses:
          - 192.168.10.2
```

Como se puede ver, le hemos configurado una IP estática y además que el servidor sea su propio DNS, ya que, al estar en una red interna, las consultas DNS se resolverán a nivel local.

Por último, aplicamos los cambios y verificamos que se hayan aplicado:

Imagen 25: Verificación de la asignación de IP en servidor web

```

ens34: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 192.168.10.2 netmask 255.255.255.0 broadcast 192.168.10.255
    ether 00:0c:29:1f:a7:fd txqueuelen 1000 (Ethernet)
    RX packets 1 bytes 342 (342.0 B)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 110 bytes 12650 (12.6 KB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

```

5.4.3 Verificación de conectividad entre máquinas

Una vez ya hemos configurado las direcciones IP estáticas de cada uno de los servidores, debemos de verificar la conectividad entre las máquinas en la red.

Procedemos a hacer un ping desde el servidor **web (192.168.10.2)** al cliente.

Como el cliente tiene dirección IP dinámica, lo que haremos será ver cual tiene. Se utilizará el comando siguiente:

Imagen 28: Ping del servidor web al cliente

```

ens34: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP gro
default qlen 1000
    link/ether 00:0c:29:5e:fd:27 brd ff:ff:ff:ff:ff:ff
    altname enp2s2
    inet 192.168.10.13/24 brd 192.168.10.255 scope global dynamic noprefixroute

```

Imagen 29: Ping desde cliente al servidor

```

dani@danicliente:~$ ping 192.168.10.2
PING 192.168.10.2 (192.168.10.2) 56(84) bytes of data:
64 bytes from 192.168.10.2: icmp_seq=1 ttl=64 time=1.41 ms
64 bytes from 192.168.10.2: icmp_seq=2 ttl=64 time=0.809 ms
64 bytes from 192.168.10.2: icmp_seq=3 ttl=64 time=1.56 ms
64 bytes from 192.168.10.2: icmp_seq=4 ttl=64 time=0.865 ms
^C
--- 192.168.10.2 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3008ms
rtt min/avg/max/mdev = 0.809/1.162/1.563/0.330 ms

```

5.5 Instalación y configuración del servidor DNS y DHCP

Para la instalación y configuración del servidor DNS para el dominio del servidor web, se utilizará la herramienta **dnsmasq**, la cual es una herramienta ligera y flexible que permite proporcionar servicios de red básicos, como **DNS** y **DHCP**.

5.5.1 Instalación de dnsmasq

Para instalar **dnsmasq**, se utilizará el siguiente comando:

Imagen 30: Comando para instalar dnsmasq

```
dani@daniserver:~$ sudo apt install dnsmasq
Leyendo lista de paquetes... Hecho
Creando árbol de dependencias... Hecho
Leyendo la información de estado... Hecho
Se instalarán los siguientes paquetes NUEVOS:
  dnsmasq
0 actualizados, 1 nuevos se instalarán, 0 para eliminar y 1 no actualizados.
Se necesita descargar 17,9 kB de archivos.
Se utilizarán 88,1 kB de espacio de disco adicional después de esta operación.
Des:1 http://es.archive.ubuntu.com/ubuntu noble/universe amd64 dnsmasq all 2.90-2build2 [17,9 kB]
Descargados 17,9 kB en 0s (128 kB/s)
Seleccionando el paquete dnsmasq previamente no seleccionado.
(Leyendo la base de datos ... 204818 ficheros o directorios instalados actualmente.)
```

Una vez instalado **dnsmasq** debemos de hacer una copia del fichero de configuración, para que, en caso de algún error en la configuración poder volver atrás, para ello implemente hacemos una copia del fichero de configuración y verificamos que se haya creado:

Imagen 31: Copia del fichero de configuración

```
dani@daniserver:~$ sudo cp /etc/dnsmasq.conf /etc/dnsmasq.conf.backup
dani@daniserver:~$ ls -l /etc/dnsmasq.conf.backup
-rw-r--r-- 1 root root 27886 may 18 17:54 /etc/dnsmasq.conf.backup
dani@daniserver:~$
```

5.5.2 Configuración del servidor DNS

Antes de comenzar con la configuración, lo ideal es dividir la configuración en varios directorios.

Para ello dnsmasq , nos ofrece una opción que permite dividir la configuración en varias partes, para tenerlo bien ordenado. Para lograr esto accedemos al fichero de configuración, y habilitamos la siguiente opción:

Imagen 32: Habilitar el directorio de fragmentos

```
# Include another lot of configuration options.
#conf-file=/etc/dnsmasq.more.conf
conf-dir=/etc/dnsmasq.d
```

Una vez activada esta opción podremos crear dentro de esta ruta diferentes ficheros de configuración, con el objetivo de que este ordenado, voy a crear el fragmento para mi servidor y verificamos que se haya creado el archivo de configuración:

Imagen 33: Creación de fichero de configuración

```
dani@daniserver:~$ sudo touch /etc/dnsmasq.d/daniserverweb.conf
dani@daniserver:~$ ls /etc/dnsmasq.d
daniserverweb.conf  README
dani@daniserver:~$
```

Una vez creado el fichero de configuración, procedemos con su edición, y escribimos la siguiente configuración para el dominio y subdominio:

Imagen 34: Configuración del servidor DNS

```
GNU nano 7.2 /etc/dnsmasq.d/daniserverweb.conf *

# Escucha solo en la dirección IP del servidor web
listen-address=192.168.10.2

# Resolución de daniserver.com y www.daniserver.com
address=/daniserver.com/192.168.10.2
address=/www.daniserver.com/192.168.10.2

# Resolución Inversa
ptr-record=2.10.168.192.in-addr.arpa,daniserver.com
ptr-record=2.10.168.192.in-addr.arpa,www.daniserver.com

# Registro de consultas DNS para monitorización y resolución de errores
log-queries
log-facility=/var/log/dnsmasq.log
```

El archivo de configuración contiene los siguientes parámetros:

- **listen-address:** esto indica que el servidor solo escuchará por la dirección IP del servidor.
- **address:** configuramos los diferentes dominios que tienen que ser resolubles en nuestro servidor, en este caso **daniserver.com** y dentro de este el servidor web, que estará alojado en **www.daniserver.com**.
- **ptr-record:** configuramos la resolución inversa del dominio.
- **log-queries:** esto activa el registro de logs y con log-facility , le especificamos la ruta donde se encontrarán los logs, esto nos ayudará a tener monitoreado el servidor DNS y poder resolver futuros problemas.

5.5.3 Verificación de resolución del dominio y logs

Por último, vamos a verificar que el dominio es resoluble, para ello vamos a utilizar el comando **dig**.

Primero verificamos el dominio **daniserver.com**:

Imagen 35: Resolución de daniserver.com

```
dani@daniserveropenvpn:~$ dig @192.168.10.2 daniserver.com

; <<>> DiG 9.18.30-0ubuntu0.24.04.2-Ubuntu <<>> @192.168.10.2 daniserver.com
; (1 server found)
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 52745
;; flags: qr aa rd ra; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 1232
;; QUESTION SECTION:
;daniserver.com.                IN      A

;; ANSWER SECTION:
daniserver.com.                0       IN      A      192.168.10.2

;; Query time: 2 msec
;; SERVER: 192.168.10.2#53(192.168.10.2) (UDP)
;; WHEN: Sun May 18 18:43:01 UTC 2025
;; MSG SIZE rcvd: 59
```

Imagen 36: Resolución de www.daniserver.com

```
dani@daniserveropenvpn:~$ dig @192.168.10.2 daniserver.com

; <<>> DiG 9.18.30-0ubuntu0.24.04.2-Ubuntu <<>> @192.168.10.2 daniserver.com
; (1 server found)
;; global options: +cmd
;; Got answer:
;; ->HEADER<- opcode: QUERY, status: NOERROR, id: 52745
;; flags: qr aa rd ra; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 1232
;; QUESTION SECTION:
; daniserver.com.                IN      A

;; ANSWER SECTION:
daniserver.com.                0      IN      A      192.168.10.2

;; Query time: 2 msec
;; SERVER: 192.168.10.2#53(192.168.10.2) (UDP)
;; WHEN: Sun May 18 18:43:01 UTC 2025
;; MSG SIZE rcvd: 59
```

Imagen 37: Resolución inversa

```
dani@daniserveropenvpn:~$ dig -x 192.168.10.2 @192.168.10.2

; <<>> DiG 9.18.30-0ubuntu0.24.04.2-Ubuntu <<>> -x 192.168.10.2 @192.168.10.2
;; global options: +cmd
;; Got answer:
;; ->HEADER<- opcode: QUERY, status: NOERROR, id: 13688
;; flags: qr aa rd ra; QUERY: 1, ANSWER: 2, AUTHORITY: 0, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 1232
;; QUESTION SECTION:
; 2.10.168.192.in-addr.arpa.     IN      PTR

;; ANSWER SECTION:
2.10.168.192.in-addr.arpa. 0      IN      PTR      www.daniserver.com.
2.10.168.192.in-addr.arpa. 0      IN      PTR      daniserver.com.

;; Query time: 1 msec
;; SERVER: 192.168.10.2#53(192.168.10.2) (UDP)
;; WHEN: Sun May 18 19:11:45 UTC 2025
;; MSG SIZE rcvd: 114
```

Por último, vamos a comprobar los logs, para ver si se van guardando en la ruta `/var/logs/dnsmasq.log`:

Imagen 38: Logs de dnsmasq

```
May 19 19:03:04 dnsmasq[5981]: query[A] daniserver.com from 192.168.10.3
May 19 19:03:04 dnsmasq[5981]: config daniserver.com is 192.168.10.2
May 19 19:03:09 dnsmasq[5981]: query[A] www.daniserver.com from 192.168.10.3
May 19 19:03:09 dnsmasq[5981]: config www.daniserver.com is 192.168.10.2
May 19 19:03:20 dnsmasq[5981]: query[PTR] 2.10.168.192.in-addr.arpa from 192.168.10.3
May 19 19:03:20 dnsmasq[5981]: config 2.10.168.192.in-addr.arpa is <PTR>
May 19 19:03:26 dnsmasq[5981]: query[A] daniserver.com from 192.168.10.3
May 19 19:03:26 dnsmasq[5981]: config daniserver.com is 192.168.10.2
May 19 19:03:26 dnsmasq[5981]: query[AAAA] daniserver.com from 192.168.10.3
May 19 19:03:26 dnsmasq[5981]: config error is REFUSED (EDE: not ready)
May 19 19:03:26 dnsmasq[5981]: query[PTR] 2.10.168.192.in-addr.arpa from 192.168.10.3
May 19 19:03:26 dnsmasq[5981]: config 2.10.168.192.in-addr.arpa is <PTR>
May 19 19:03:34 dnsmasq[5981]: query[A] www.daniserver.com from 192.168.10.3
May 19 19:03:34 dnsmasq[5981]: config www.daniserver.com is 192.168.10.2
May 19 19:03:34 dnsmasq[5981]: query[AAAA] www.daniserver.com from 192.168.10.3
May 19 19:03:34 dnsmasq[5981]: config error is REFUSED (EDE: not ready)
May 19 19:03:34 dnsmasq[5981]: query[PTR] 2.10.168.192.in-addr.arpa from 192.168.10.3
May 19 19:03:34 dnsmasq[5981]: config 2.10.168.192.in-addr.arpa is <PTR>
```

Como se ve en la imagen, los logs sí que se van guardando.

5.5.4 Creación de servidor DHCP con dnsmasq

Para crear el servidor DHCP, lo primero es crear un archivo de configuración, el cual estará ubicado en la ruta `/etc/dnsmasq.d/dhcp.conf`:

Imagen 39: Creación de fichero de configuración de DHCP

```
dani@daniserver:~$ sudo touch /etc/dnsmasq.d/dhcp.conf
[sudo] password for dani:
dani@daniserver:~$ ls /etc/dnsmasq.d/
daniserverweb.conf  dhcp.conf  README
dani@daniserver:~$
```

Ahora editamos el archivo y ponemos la siguiente configuración:

Imagen 40: Configuración del fichero

```
GNU nano 7.2 /etc/dnsmasq.d/dhcp.conf *

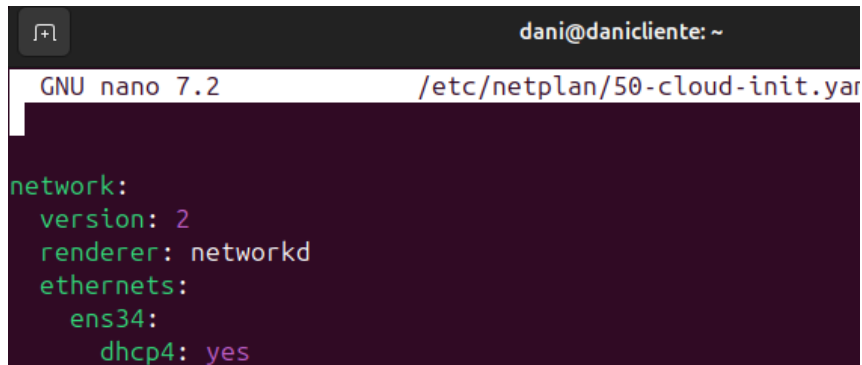
interface=ens34
dhcp-range=192.168.10.10,192.168.10.100,24h
dhcp-option=3,192.168.10.2
dhcp-option=6,192.168.10.2
dhcp-authoritative
no-resolv
```

La configuración tiene los siguientes parámetros:

- **interface:** indicamos la interfaz por la que funcionará el DHCP.
- **dhcp-range:** especificamos el rango de IPs y su tiempo de concesión.
- **dhcp-authoritative:** indicamos que el servidor DHCP es el principal de la red.
- **dhcp-option:** configuramos el gateway y el DNS, el numero 3 pertenece al parámetro del gateway y el 6 al DNS.
- **no-resolv:** evitamos que utilice resolvers externos como **resolv.conf**

Ahora vamos al cliente y editamos el archivo **50-cloud-init.yaml** y ponemos la interfaz ens34 por DHCP:

Imagen 41: Configuración del cliente para DHCP

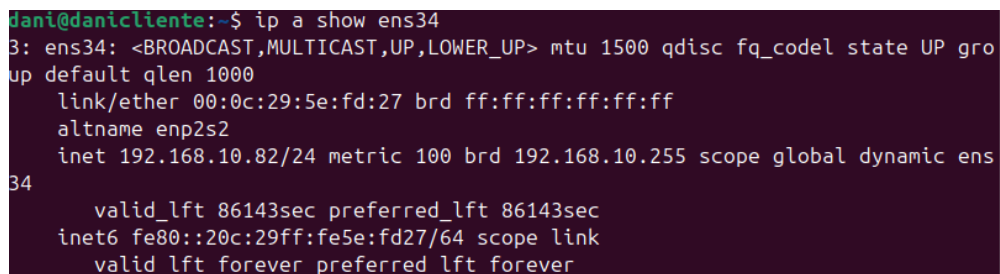


```
dani@danicliente: ~
GNU nano 7.2 /etc/netplan/50-cloud-init.yaml

network:
  version: 2
  renderer: networkd
  ethernets:
    ens34:
      dhcp4: yes
```

Ahora aplicamos cambios y verificamos que se le haya asignado IP al cliente:

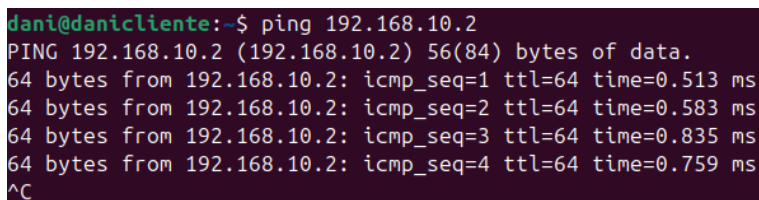
Imagen 42: Verificación de asignación de IP



```
dani@danicliente:~$ ip a show ens34
3: ens34: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP gro
up default qlen 1000
    link/ether 00:0c:29:5e:fd:27 brd ff:ff:ff:ff:ff:ff
    altname enp2s2
    inet 192.168.10.82/24 metric 100 brd 192.168.10.255 scope global dynamic ens
34
        valid_lft 86143sec preferred_lft 86143sec
    inet6 fe80::20c:29ff:fe5e:fd27/64 scope link
        valid_lft forever preferred_lft forever
```

Una vez se ha asignado la IP verificamos con un ping que se conecte con el servidor:

Imagen 43: Ping con el servidor



```
dani@danicliente:~$ ping 192.168.10.2
PING 192.168.10.2 (192.168.10.2) 56(84) bytes of data.
64 bytes from 192.168.10.2: icmp_seq=1 ttl=64 time=0.513 ms
64 bytes from 192.168.10.2: icmp_seq=2 ttl=64 time=0.583 ms
64 bytes from 192.168.10.2: icmp_seq=3 ttl=64 time=0.835 ms
64 bytes from 192.168.10.2: icmp_seq=4 ttl=64 time=0.759 ms
^C
```

5.6 Instalación y configuración del servidor web

5.6.1 Instalación de Apache2

Para instalar Apache2, utilizaremos el siguiente comando:

Imagen 39: Instalación de Apache2

```
dani@daniserver:~$ sudo apt install apache2
Leyendo lista de paquetes... Hecho
Creando árbol de dependencias... Hecho
Leyendo la información de estado... Hecho
Se instalarán los siguientes paquetes adicionales:
  apache2-bin apache2-data apache2-utils libapr1t64 libaprutil1-dbd-sqlite3
  libaprutil1-ldap libaprutil1t64
Paquetes sugeridos:
  apache2-doc apache2-suexec-pristine | apache2-suexec-custom
Se instalarán los siguientes paquetes NUEVOS:
  apache2 apache2-bin apache2-data apache2-utils libapr1t64
  libaprutil1-dbd-sqlite3 libaprutil1-ldap libaprutil1t64
0 actualizados, 8 nuevos se instalarán, 0 para eliminar y 17 no actualizados.
Se necesita descargar 1.900 kB de archivos.
Se utilizarán 7.455 kB de espacio de disco adicional después de esta operación.
```

Una vez ya tengamos instalado apache2, procedemos a comprobar que el servicio esté activo:

Imagen 40: Verificación de estado del servicio apache2

```
● apache2.service - The Apache HTTP Server
   Loaded: loaded (/usr/lib/systemd/system/apache2.service; enabled; preset: enabled)
   Active: active (running) since Sat 2025-05-24 14:05:15 CEST; 4min 16s ago
     Docs: https://httpd.apache.org/docs/2.4/
    Main PID: 4706 (apache2)
      Tasks: 55 (limit: 4545)
     Memory: 5.3M (peak: 5.6M)
        CPU: 152ms
    CGroup: /system.slice/apache2.service
            └─4706 /usr/sbin/apache2 -k start
              └─4708 /usr/sbin/apache2 -k start
                └─4709 /usr/sbin/apache2 -k start

may 24 14:05:15 daniserver systemd[1]: Starting apache2.service - The Apache HTTP Server...
may 24 14:05:15 daniserver apachectl[4705]: AH00558: apache2: Could not reliably determine the
may 24 14:05:15 daniserver systemd[1]: Started apache2.service - The Apache HTTP Server.
```

5.6.2 Habilitar módulo SSL y generación de certificado

Para habilitar el módulo SSL en Apache hacemos uso del siguiente comando:

Imagen 41: Habilitar módulo SSL y sitio por defecto SSL

```
dani@dani-server:~$ sudo a2enmod ssl
Considering dependency mime for ssl:
Module mime already enabled
Considering dependency socache_shmcb for ssl:
Enabling module socache_shmcb.
Enabling module ssl.
See /usr/share/doc/apache2/README.Debian.gz on how to configure SSL and create s
elf-signed certificates.
To activate the new configuration, you need to run:
    systemctl restart apache2
dani@dani-server:~$ sudo a2ensite default-ssl.conf
Enabling site default-ssl.
```

Ahora para aplicar los cambios y que el módulo SSL, se active, debemos de reiniciar Apache:

Imagen 42: Reinicio del servicio apache2

```
dani@dani-server:~$ sudo systemctl restart apache2
dani@dani-server:~$ sudo systemctl status apache2

● apache2.service - The Apache HTTP Server
   Loaded: loaded (/usr/lib/systemd/system/apache2.service; enabled; preset: enabled)
   Active: active (running) since Sat 2025-05-24 14:25:11 CEST; 5s ago
     Docs: https://httpd.apache.org/docs/2.4/
   Process: 13995 ExecStart=/usr/sbin/apachectl start (code=exited, status=0/SUCCESS)
  Main PID: 13998 (apache2)
    Tasks: 55 (limit: 4545)
   Memory: 6.1M (peak: 6.4M)
      CPU: 39ms
   CGroup: /system.slice/apache2.service
           └─13998 /usr/sbin/apache2 -k start
             └─14000 /usr/sbin/apache2 -k start
               └─14001 /usr/sbin/apache2 -k start
```

Ahora procedemos con la generación del certificado SSL autofirmado , para ello, lo primero que haremos será crear un directorio, para almacenar el certificado y la clave:

Imagen 43: Creación de directorio para certificado y clave

```
dani@dani-server:~$ sudo mkdir /etc/apache2/ssl
dani@dani-server:~$
```

Ahora generamos el certificado, el comando para hacerlo es el siguiente:

Imagen 44: Comando para generar certificado y clave

```
dani@daniserver:~$ sudo openssl req -x509 -nodes -days 365 -newkey rsa:2048 -keyout /etc/apache2/ssl/daniserverweb.key -out /etc/apache2/ssl/daniserverweb.crt
```

Los parámetros que tiene el comando son los siguientes:

- **req -x509:** especifica que vamos a usar la administración de firma de certificados x.509.
- **-nodes:** omite la opción de protección de clave con una contraseña, ya que nos interesa que Apache lea el archivo.
- **-days 365:** establece la duración de validez del certificado, pasado un año habría que renovarlo.
- **-newkey rsa:2048:** especificamos que queremos generar una clave y certificado al mismo tiempo. La parte de rsa, nos indica que cree un clave rsa de 2048 bits de extensión.
- **-keyout:** le indica a OpenSSL dónde debe colocar el archivo de clave privada.
- **-out:** le indica a OpenSSL donde colocar el certificado.

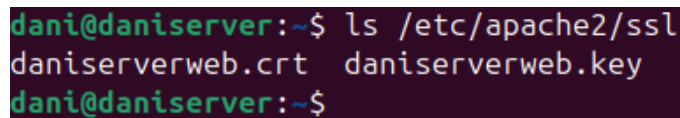
Una vez ejecutamos el comando tenemos que completar los siguientes campos, el más importante es el **CN**, ya que dentro de este tenemos que indicar el nombre de dominio de nuestra página web, en nuestro caso www.daniserver.com:

Imagen 45: Comando para generar certificado y clave

```
Country Name (2 letter code) [AU]:ES
State or Province Name (full name) [Some-State]:
Locality Name (eg, city) []:
Organization Name (eg, company) [Internet Widgits Pty Ltd]:
Organizational Unit Name (eg, section) []:
Common Name (e.g. server FQDN or YOUR name) []:www.daniserver.com
Email Address []:admin@daniserver.com
```

Una vez hecho esto, ya tendríamos los certificados generados ahora debemos de verificar que estén en la ruta especificada:

Imagen 46: Verificación de creación de certificados en la ruta especificada

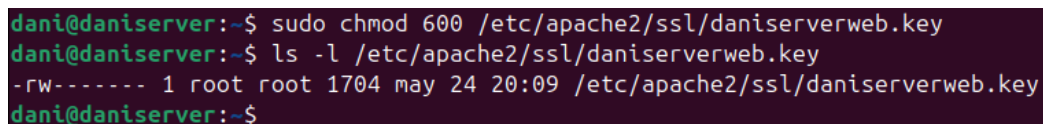


```
dani@daniserver:~$ ls /etc/apache2/ssl
daniserverweb.crt  daniserverweb.key
dani@daniserver:~$
```

Como se puede ver se han generado las dos claves, es decir, la privada llamada **daniserverweb.key** y el certificado de **daniserverweb.crt**

Después de comprobar que están generados los certificados, debemos de dar permisos seguros a la clave privada, es decir, configurarlo para que solo el usuario root tenga permisos de lectura y escritura sobre la clave:

Imagen 47: Permisos de clave privada



```
dani@daniserver:~$ sudo chmod 600 /etc/apache2/ssl/daniserverweb.key
dani@daniserver:~$ ls -l /etc/apache2/ssl/daniserverweb.key
-rw----- 1 root root 1704 may 24 20:09 /etc/apache2/ssl/daniserverweb.key
dani@daniserver:~$
```

5.6.3 Configuración del Virtual Host

Para configurar el Virtual Hosts, debemos de crear un fichero de configuración dentro de la ruta **/etc/apache2/sites-available**.

De esta forma Apache podrá leer nuestra configuración del Virtual Hosts de la página:

Imagen 48: Creación de fichero de configuración para el Virtual Host

```
dani@daniserver:/etc/apache2/sites-available$ ls
000-default.conf  default-ssl.conf
dani@daniserver:/etc/apache2/sites-available$ sudo touch daniserver.conf
dani@daniserver:/etc/apache2/sites-available$ ls
000-default.conf  daniserver.conf  default-ssl.conf
dani@daniserver:/etc/apache2/sites-available$
```

Como se puede ver hay tres páginas, las dos que hay, debemos de deshabilitarlas posteriormente, para que no entre en conflicto con los otros sitios y cargue el que queremos.

Antes de empezar con la configuración del Virtual Host debemos de activar los **headers**, ya que para mejorar la seguridad del servidor web configuraremos alguno de ellos.

Para activar el módulo **headers** debemos de introducir el siguiente comando:

Imagen 49: Activación del módulo headers para configuración de encabezados

```
dani@daniserver:~$ sudo a2enmod headers
[sudo] password for dani:
Enabling module headers.
To activate the new configuration, you need to run:
systemctl restart apache2
```

Por último, reiniciamos Apache para aplicar los cambios.

Una vez hecho todo esto, procedemos con la configuración del Virtual Host. La configuración el Virtual Host para el puerto **80 (HTTP)**, es la siguiente:

Imagen 50: Configuración Virtual Host en el puerto 80 (HTTP)

```
GNU nano 7.2                                daniserver.conf
<VirtualHost 192.168.10.2:80>
  ServerName www.daniserver.com
  ServerAlias daniserver.com
  Redirect permanent / https://www.daniserver.com/
</VirtualHost>
```

La configuración contiene lo siguiente:

- **VirtualHost 192.168.10.2:80:** esto indica que responde a las solicitudes HTTP en la dirección IP establecida y puerto 80.
- **ServerName:** nombre del servidor principal.
- **ServerAlias:** nombre alternativo para el sitio web.
- **Redirect permanent / https://www.daniserver.com/:** redirige todas las solicitudes que lleguen a este servidor vía HTTP a HTTPS.

Ahora procedemos con la configuración para el puerto **443(HTTPS)**. La configuración es la siguiente:

Imagen 51: Configuración Virtual Host en el puerto 443 (HTTPS)

```
GNU nano 7.2                                daniserver.conf
<VirtualHost 192.168.10.2:443>
  ServerName www.daniserver.com
  ServerAlias daniserver.com

  DocumentRoot /var/www/daniserver
  <Directory /var/www/daniserver>
    Options -Indexes +FollowSymLinks
    AllowOverride All
    Require all granted
  </Directory>

  SSLEngine on
  SSLCertificateFile /etc/apache2/ssl/daniserverweb.crt
  SSLCertificateKeyFile /etc/apache2/ssl/daniserverweb.key

  SSLProtocol TLSv1.3
  SSLHonorCipherOrder on

  Header always set Strict-Transport-Security "max-age=31536000; includeSubDomains; preload"
  Header always set X-Content-Type-Options "nosniff"
  Header always set X-Frame-Options "DENY"

  ErrorLog /var/log/daniserverweb/error.log
  CustomLog /var/log/daniserverweb/access.log combined
</VirtualHost>
```

La configuración tiene los siguientes parámetros:

- **VirtualHost 192.168.10.2:443:** responde las solicitudes HTTPS en la dirección establecida.

- **DocumentRoot /var/www/daniserver:** indicamos el directorio donde se encontrarán los archivos de la web.
- **Directory /var/www/daniserver:** dentro de este bloque., configuramos el comportamiento del directorio para el sitio web.

Dentro de este bloque hemos llevado a cabo la configuración de los siguientes parámetros:

1. **Options -Indexes +FollowSymLinks:** permite listar archivos y seguir enlaces simbólicos.
 2. **AllowOverride All:** permite que archivos .htaccess sobrescriban configuraciones.
 3. **Require all granted:** permite acceso a todos los visitantes de la web.
- **SSLEngine on:** activa el motor ssl para el virtual host.
 - **SSLCertificateFile:** especificamos la ruta hacia el archivo de certificado SSL.
 - **SSLCertificateKeyFile:** especificamos la ruta hacia la clave privada correspondiente.
 - **SSLProtocol TLSv1.3:** forzamos el uso del protocolo TLSv1.3, que es mucho más seguro que versiones anteriores del protocolo.
 - **SSLHonorCipherOrder on:** hacemos que el servidor priorice su propia lista de cifrados.
 - **Header always set Strict-Transport-Security:** obligamos a que los navegadores utilicen siempre HTTPS, y le hemos configurado los siguientes parámetros a la cabecera:
 1. **max-age=31536000:** tiempo que el navegador forzará HTTPS, en este caso un año.
 2. **IncludeSubDomains:** aplicamos también a subdominios.
 3. **preload:** indicamos que nuestro dominio debe incluirse en la lista de precarga HSTS. Esta lista contiene los dominios que deben ser accedidos si o si por HTTPS.
 - **Header always set X-Content-Type-Options "nosniff":** evita que el navegador interprete el tipo de contenido que recibe. Gracias esto evitamos ataques de ejecución de código malicioso.

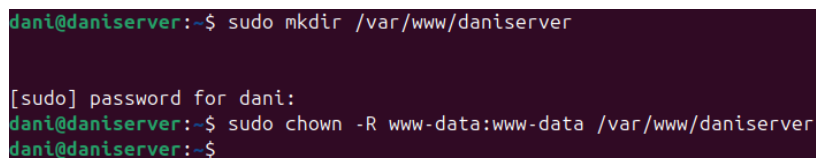
- **Header always set X-Frame-Options "DENY"**: evita que la página sea mostrada en un frame en ningún caso, lo que evita ataques de clickjacking.
- **ErrorLog /var/log/daniserverweb/error.log**: definimos la ubicación donde almacenaremos los logs de errores.
- **CustomLog /var/log/daniserverweb/access.log combined**: definimos donde se encontrarán los registros de acceso y con la opción **combined**, mostramos la IP.

Una vez configurado el VirtualHost , sabiendo las rutas del directorio y de los logs debemos de crearlas y configurar los permisos de forma correcta para que todo funcione correctamente.

Primero vamos a crear la ruta **/var/www/daniserver** y configurar los grupos y permisos necesarios para el directorio de contenido del sitio web.

Lo primero que haremos será crear el directorio y cambiar el propietario del directorio y su contenido al usuario por defecto de Apache, que es **www-data**:

Imagen 51: Creación de directorio de contenido y cambio de propietario a usuario www-data



```
dani@daniserver:~$ sudo mkdir /var/www/daniserver
[sudo] password for dani:
dani@daniserver:~$ sudo chown -R www-data:www-data /var/www/daniserver
dani@daniserver:~$
```

Con esto nos aseguramos de que Apache sea dueño de los archivos, permitiendo que pueda leerlos sin problemas.

Ahora debemos cambiar los permisos, pero para configurarlos de forma segura utilizaremos el comando **find**. Si aplicáramos permisos de forma generalizada con **chmod**, estaríamos asignando los mismos permisos tanto a archivos como a directorios, lo cual podría comprometer la seguridad del servidor. Por ejemplo, el archivo index.html solo necesita ser leído por Apache, no requiere permisos de ejecución ni escritura. Por lo tanto, no es recomendable usar **chmod** de forma global en los permisos. En su lugar,

utilizaremos **find** para diferenciar y aplicar los permisos adecuados de forma específica a archivos y directorios.

Dicho esto, procedemos con la asignación de permisos necesarios y seguros:

Imagen 52: Asignación de permisos a /var/www/daniserver



```
dani@daniserver:~$ sudo find /var/www/daniserver -type d -exec chmod 755 {} \;  
dani@daniserver:~$ sudo find /var/www/daniserver -type f -exec chmod 644 {} \;  
dani@daniserver:~$
```

Cada uno de los comandos hace lo siguiente:

- **`sudo find /var/www/daniserver -type d -exec chmod 755 {} \;`**

Este comando establece en la ruta especificada los permisos siguientes para los directorios. En los que el propietario tiene permisos de lectura, escritura y ejecución y los grupos y otros tienen permisos de lectura y ejecución.

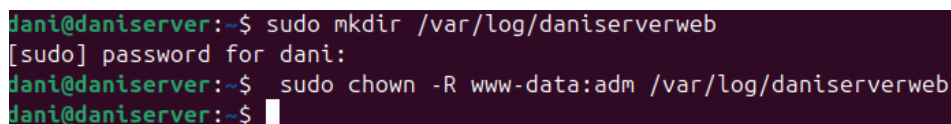
- **`find /var/www/daniserver -type f -exec chmod 644 {} \;`**

Este comando establece en la ruta especificada los permisos siguientes para los ficheros. En los que el propietario tiene permisos de lectura y escritura y los grupos y otros tienen permisos de lectura únicamente.

Una vez hemos configurado los permisos para el directorio de contenido del sitio web, debemos asegurarnos que apache puede acceder y escribir dentro de los directorios creados anteriormente para los logs.

Para conseguir esto, lo primero que haremos será crear el directorio y cambiar el propietario:

Imagen 53: Creación de directorio de contenido y cambio de propietario a usuario www-data y grupo adm(admin)



```
dani@daniserver:~$ sudo mkdir /var/log/daniserverweb  
[sudo] password for dani:  
dani@daniserver:~$ sudo chown -R www-data:adm /var/log/daniserverweb  
dani@daniserver:~$
```

Ahora establecemos los permisos necesarios:

Imagen 54: Asignación de permisos a /var/www/daniserver

```
dani@daniserver:~$ sudo chmod 750 /var/log/daniserverweb
dani@daniserver:~$
```

Los permisos que hemos configurado son para que el propietario tenga todos los permisos, el grupo tenga permisos de lectura y ejecución y el resto nada.

Una vez configurado cada uno de los permisos, ya tendríamos todo preparado para activar el sitio web, pero antes de esto debemos de desactivar los sitios web por defecto para evitar conflictos.

Para ello vamos a desactivar las dos que tenemos por defecto y activamos la que hemos configurado:

Imagen 54: Asignación de permisos a /var/www/daniserver

```
dani@daniserver:~$ sudo a2dissite 000-default.conf
Site 000-default disabled.
To activate the new configuration, you need to run:
  systemctl reload apache2
dani@daniserver:~$ sudo a2dissite default-ssl.conf
Site default-ssl disabled.
To activate the new configuration, you need to run:
  systemctl reload apache2
dani@daniserver:~$ sudo a2ensite daniserver.conf
Enabling site daniserver.
To activate the new configuration, you need to run:
  systemctl reload apache2
```

Una vez hecho esto tendríamos que reiniciar Apache y ya lo tendríamos todo configurado y listo.

Ahora si intentamos acceder desde el cliente debería de dejarnos entrar en la página, pero debido a la configuración no nos va a dejar ver los directorios, solo nos dejará ver la página cuando creemos un archivo .html, .php etc.

Es decir que si no tenemos configurada un archivo para mostrar contenido el cliente no podrá acceder, le pondrá acceso denegado hasta que configuremos la página web en el directorio **/var/www/daniserver**.

5.7 Creación básica de página web para pruebas de acceso

Para verificar que el servidor funciona correctamente crearemos una web sencilla con HTML y PHP.

La página va a ser básica y permitirá detectar si un número es decimal o no, para ello tenemos el siguiente código en HTML y PHP:

Imagen 55: Creación del formulario HTML

```
<!DOCTYPE html>
<!--Daniel Monge-->
<html lang="es">
<head>
  <meta charset="UTF-8">
  <title>Numero decimal</title>
</head>
<body>
  <h2>Introducir un número decimal:</h2>
  <form action="decimal.php" method="post">
    <label for="numero">Número decimal:</label>
    <input type="number" id="numero" name="numero" step="any" required>
    <br><br>
    <input type="submit" value="Enviar">
  </form>
</body>
</html>
```

El archivo html tiene lo siguiente:

- **form:** con esta etiqueta hacemos que los datos se envíen al archivo decimal.php para poder procesarlos.
- **label:** asociamos la etiqueta está asociado al campo con id número.
- **input:** es el campo que recibe el número con su id, nombre y step que es para que el usuario pueda introducir cualquier número decimal.

Ahora vamos con el archivo .php , el código sería el siguiente:

Imagen 56: Creación del archivo php para procesamiento de los datos

```

<?php
//Daniel Monge
if ($_SERVER["REQUEST_METHOD"] == "POST") {
    $numero = $_POST['numero'] ?? '';

    echo "<!DOCTYPE html>
    <html lang='es'>
    <head>
        <meta charset='UTF-8'>
        <title>Resultado</title>
    </head>
    <body>
        <h2>Has introducido el número: $numero</h2>
        <p><a href='formulario.html'>Volver al formulario</a></p>
    </body>
    </html>";
} else {
    header("Location: formulario.html");
    exit;
}
?>

```

Lo primero que hacemos es verificar que la petición se por método POST, después almacenamos el número que el usuario ha escrito y si no existe le asignamos cadena vacía para que no de error.

Ahora lo que hacemos es añadir un HTML para que el usuario pueda ver el número que eligió y poder volver al formulario si así lo desea.

Si el método no es POST, le redirigimos automáticamente a formulario.html.

Una vez hecho esto, antes de poner los dos archivos debemos de instalar el módulo PHP en apache para que pueda procesarlo:

Imagen 57: Instalación de módulo PHP

```

dani@dani-server:~$ sudo apt install php-common libapache2-mod-php php-cli

Leyendo lista de paquetes... Hecho
Creando árbol de dependencias... Hecho
Leyendo la información de estado... Hecho
Se instalarán los siguientes paquetes adicionales:
  libapache2-mod-php8.3 php8.3-cli php8.3-common php8.3-opcache php8.3-readline
Paquetes sugeridos:
  php-pear
Se instalarán los siguientes paquetes NUEVOS:
  libapache2-mod-php libapache2-mod-php8.3 php-cli php-common php8.3-cli php8.3-
common php8.3-opcache php8.3-readline
0 actualizados, 8 nuevos se instalarán, 0 para eliminar y 26 no actualizados.
Se necesita descargar 4.912 kB de archivos.
Se utilizarán 22,3 MB de espacio de disco adicional después de esta operación.
¿Desea continuar? [S/n] s

```

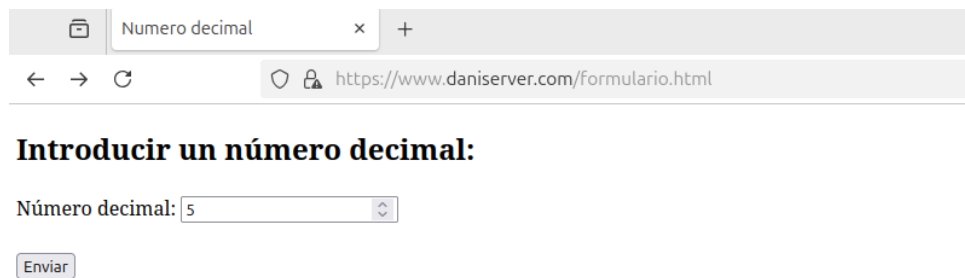
Ahora añadimos las dos páginas creadas al escritorio y las copiamos en la ruta `/var/www/daniserver`:

Imagen 58: Copia de archivos a `/var/www/daniserver`

```
dani@daniserver:~/Desktop$ ls
decimal.php  formulario.html
dani@daniserver:~/Desktop$ sudo cp formulario.html /var/www/daniserver
dani@daniserver:~/Desktop$ sudo cp decimal.php /var/www/daniserver
dani@daniserver:~/Desktop$ cd /var/www/daniserver
dani@daniserver:/var/www/daniserver$ ls
decimal.php  formulario.html  index.html
dani@daniserver:/var/www/daniserver$ sudo rm -r index.html
dani@daniserver:/var/www/daniserver$ ls
decimal.php  formulario.html
dani@daniserver:/var/www/daniserver$
```

Ahora con el cliente entramos a la página y verificamos que funciona:

Imagen 59: Acceso a formulario HTML



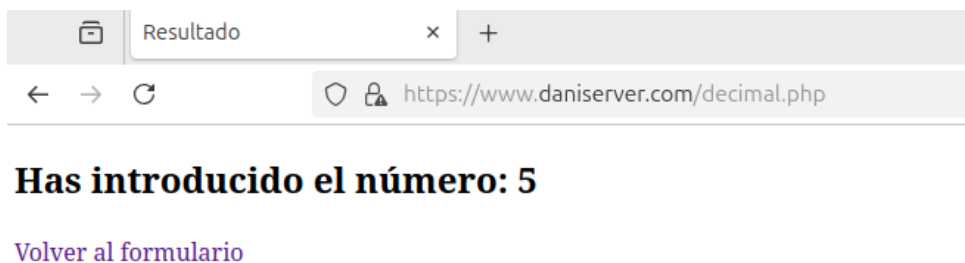
Numero decimal x +

← → ↻ https://www.daniserver.com/Formulario.html

Introducir un número decimal:

Número decimal:

Imagen 60: Comportamiento del archivo PHP una vez introducido el dato



Resultado x +

← → ↻ https://www.daniserver.com/decimal.php

Has introducido el número: 5

[Volver al formulario](#)

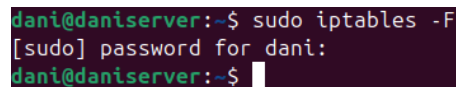
5.8 Configuración de firewall con iptables

Ahora para mejorar la seguridad del servidor configuraremos una serie de reglas con iptables.

Antes de realizar cualquier configuración debemos instalar **iptables** con el comando **sudo apt install iptables -y**.

Una vez instalado, lo primero que haremos será limpiar todas las reglas existentes y personalizadas creadas anteriormente para asegurarnos que lo desde cero y no hay reglas ya configuradas:

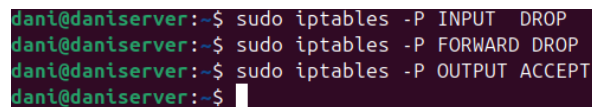
Imagen 60: Eliminación de reglas existentes



```
dani@dani-server:~$ sudo iptables -F
[sudo] password for dani:
dani@dani-server:~$
```

Ahora establecemos las políticas predeterminadas:

Imagen 61: Establecer políticas predeterminadas



```
dani@dani-server:~$ sudo iptables -P INPUT DROP
dani@dani-server:~$ sudo iptables -P FORWARD DROP
dani@dani-server:~$ sudo iptables -P OUTPUT ACCEPT
dani@dani-server:~$
```

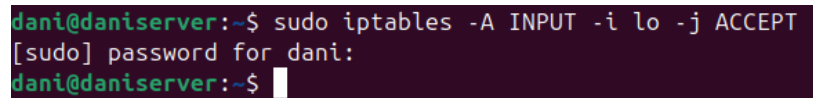
Las reglas hacen lo siguiente:

- **sudo iptables -P INPUT DROP:** todos los paquetes entrantes que no coincidan con las reglas que configuraremos posteriormente serán descartados.
- **iptables -P FORWARD DROP:** con esta regla hacemos que el servidor por defecto no reenvíe ningún paquete, es decir solo permitirá el tráfico que llegue directamente al servidor.
- **iptables -P OUTPUT ACCEPT:** permitimos todo el tráfico saliente, es decir que el servidor se podrá conectar a internet sin restricciones. (esta regla la configuramos en caso de que el servidor vaya a salir a internet, ya que si estamos

en un entorno interno no hará falta, pero si estamos configurándolo para hacer pruebas y después sacarlo a la red, entonces sí).

Ahora habilitamos reglas para el tráfico local:

Imagen 62: Regla para el tráfico local

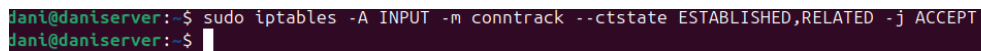


```
dani@daniserver:~$ sudo iptables -A INPUT -i lo -j ACCEPT
[sudo] password for dani:
dani@daniserver:~$
```

Con esto permitimos todo el tráfico interno del servidor, necesario para cualquier servicio local.

Ahora debemos de permitir las conexiones ya establecidas y que relacionadas, para ello aplicamos las siguientes dos reglas:

Imagen 63: Permitir conexiones establecidas y relacionadas



```
dani@daniserver:~$ sudo iptables -A INPUT -m conntrack --ctstate ESTABLISHED,RELATED -j ACCEPT
dani@daniserver:~$
```

Esta regla permite identificar a iptables el estado de una conexión. Esta regla se aplicará solo a los paquetes cuyas conexiones sean **ESTABLISHED** (una conexión ya establecida y permitida anteriormente) y **RELATED** (paquetes relacionados con una conexión ya establecida). Esta regla permitirá estas conexiones y evitarán que se cierren.

Por último, debemos permitir el tráfico entrante por el puerto **80(HTTP)**, puerto **443(HTTPS)** y **53(DNS)** para que Apache funcione correctamente:

Imagen 64: Permitir puertos web y DNS

```
dani@daniserver:~$ sudo iptables -A INPUT -p tcp --dport 53 -m conntrack --ctstate NEW -j ACCEPT
dani@daniserver:~$ sudo iptables -A INPUT -p tcp --dport 80 -m conntrack --ctstate NEW -j ACCEPT
dani@daniserver:~$ sudo iptables -A INPUT -p tcp --dport 443 -m conntrack --ctstate NEW -j ACCEPT
dani@daniserver:~$
```

Estas reglas permiten nuevas conexiones en los puertos 53(DNS), 80(HTTP) y 443(HTTPS).

Una vez configurado todas estas reglas, debemos de asegurarnos de que estén todas bien configuradas, para ello listamos las reglas y verificamos que estén todas configuradas:

Imagen 65: Listar reglas iptables

```
dani@daniserver:~$ sudo iptables -L
Chain INPUT (policy DROP)
target     prot opt source                destination
ACCEPT     all  --  anywhere               anywhere
ACCEPT     all  --  anywhere               anywhere
ACCEPT     tcp  --  anywhere               anywhere       ctstate RELATED,ESTABLISHED
ACCEPT     tcp  --  anywhere               anywhere       tcp dpt:domain ctstate NEW
ACCEPT     tcp  --  anywhere               anywhere       tcp dpt:http ctstate NEW
ACCEPT     tcp  --  anywhere               anywhere       tcp dpt:https ctstate NEW

Chain FORWARD (policy DROP)
target     prot opt source                destination

Chain OUTPUT (policy ACCEPT)
target     prot opt source                destination
dani@daniserver:~$
```

Una vez verificamos que están todas las reglas debemos de hacer los cambios persistentes, para ello instalamos **iptables-persistent**.

Una vez instalado escribimos el siguiente comando:

Imagen 66: Guardar reglas iptables actuales

```
dani@daniserver:~$ sudo netfilter-persistent save
run-parts: executing /usr/share/netfilter-persistent/plugins.d/15-ip4tables save
run-parts: executing /usr/share/netfilter-persistent/plugins.d/25-ip6tables save
dani@daniserver:~$
```

Ahora, verificaremos con nmap que están abiertos los puertos que queremos, para ello lo primero es instalar **nmap**:

Imagen 66: Instalación de nmap

```
dani@daniserver:~$ sudo apt install nmap
Leyendo lista de paquetes... Hecho
Creando árbol de dependencias... Hecho
Leyendo la información de estado... Hecho
Se instalarán los siguientes paquetes adicionales:
  libblas3 liblinear4 libssh2-1t64 nmap-common
Paquetes sugeridos:
  liblinear-tools liblinear-dev ncat ndiff zenmap
Se instalarán los siguientes paquetes NUEVOS:
  libblas3 liblinear4 libssh2-1t64 nmap nmap-common
0 actualizados, 5 nuevos se instalarán, 0 para eliminar y 26 no actualizados.
Se necesita descargar 6.286 kB de archivos.
Se utilizarán 27,4 MB de espacio de disco adicional después de esta operación.
¿Desea continuar? [S/n] s
```

Una vez instalado verificamos los puertos que están abiertos y verificamos que estén abiertos los que hemos configurado:

Imagen 66: Escaneo de puertos y verificación

```
dani@daniserver:~$ nmap localhost
Starting Nmap 7.94SVN ( https://nmap.org ) at 2025-06-01 15:19 CEST
Nmap scan report for localhost (127.0.0.1)
Host is up (0.0015s latency).
Not shown: 997 closed tcp ports (conn-refused)
PORT      STATE SERVICE
53/tcp    open  domain
80/tcp    open  http
443/tcp   open  https

Nmap done: 1 IP address (1 host up) scanned in 0.25 seconds
dani@daniserver:~$
```

Como se puede ver todos los puertos están abiertos.

Y con esto ya tendríamos las reglas básicas de iptables configuradas.

6. Conclusión

Durante el desarrollo de todo este proyecto hemos podido darnos cuenta de la gran importancia que tiene la seguridad en los servidores hoy en día. En entornos empresariales, donde cada vez hay más servidores configurados y conectados a Internet, también aumenta la cantidad de amenazas. Por ello, una correcta configuración de seguridad en los servidores se vuelve fundamental.

En este proyecto he podido observar y aprender de forma más avanzada sobre la importancia de la seguridad, especialmente en la configuración del servidor Apache, así como en la aplicación de medidas de protección mediante iptables, una herramienta clave para controlar el tráfico de red y limitar los accesos.

También hemos configurado certificados SSL autofirmados, los cuales, aunque no están validados por una entidad certificadora, siguen siendo útiles para establecer conexiones cifradas entre cliente y servidor. A pesar de no contar con una firma oficial, ofrecen seguridad en la transmisión de datos, ya que el cifrado sigue funcionando correctamente.

En conclusión, este proyecto ha sido muy interesante para profundizar en el campo de la seguridad en servidores, permitiéndome integrar diversas herramientas y buenas prácticas de seguridad que mejoran la seguridad en servidores.

7. Webgrafía

Instalación de Ubuntu junto con entorno de escritorio

https://documentation.ubuntu.com/_/downloads/server/en/latest/pdf/

(Paginas 9 y 10)

<https://phoenixnap.es/kb/how-to-install-a-gui-on-ubuntu>

Configuración de IP estática en Ubuntu

<https://netplan.readthedocs.io/en/stable/examples/#how-to-configure-a-static-ip-address-on-an-interface>

DNS con dnsmasq

<https://help.ubuntu.com/community/Dnsmasq>

<https://serverfault.com/questions/136332/setting-up-dnsmasq-for-a-local-network>

DHCP con dnsmasq

<https://sio2sio2.github.io/doc-linux/06.infraestructura/02.dhcp/03.dnsmasq.html>

Servidor web y certificados SSL

<https://www.digitalocean.com/community/tutorials/how-to-create-a-self-signed-ssl-certificate-for-apache-in-ubuntu-20-04-es>

https://httpd.apache.org/docs/2.4/ssl/ssl_howto.html

Información sobre TLS

<https://www.cloudflare.com/es-es/learning/ssl/why-use-tls-1.3/>

Cabeceras de seguridad en servidor web

<https://developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Headers/Strict-Transport-Security>

<https://developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Headers/X-Content-Type-Options>

https://developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Headers/X-Frame-Options#allow-from_origin

Permisos Apache

<https://usuariodebian.blogspot.com/2020/09/apache-permisos-usuario-y-grupo-www-data.html>

Instalación del módulo PHP

<https://www.php.net/manual/es/install.unix.debian.php>

Iptables

<https://wiki.archlinux.org/title/Iptables>

<https://www.digitalocean.com/community/tutorials/how-to-list-and-delete-iptables-firewall-rules-es>

<https://www.redeszone.net/tutoriales/seguridad/iptables-firewall-linux-configuracion/>